

ECE598: Generative Modeling with Diffusion and Flow Matching Models — Homework 1

Prof. Zhizhen Jane Zhao
TA: Yichi Zhang

Instructions

- Homework is due **Friday, Oct 17th, 11:59PM CDT**.
- Everyone must submit individually at gradescope under **hw1**.
- We reserve the right to reduce the auto-graded score for **hw1** if we detect funny business (e.g., your solution lacks any algorithm and hard-codes answers you obtained from someone else, or simply via trial-and-error with the autograder).
- When submitting to **hw1**, only upload **hw1.py**. Additional files will be ignored.

Task Overview

In this homework you will implement the core components of a diffusion model. We provide the datasets, model architecture, training loop, evaluation metrics, and plotting code. Your task is to complete several missing functions in the `GaussianDiffusion` class.

After completing the implementation, you will be able to:

- Train a diffusion model on simple 1D and 2D toy datasets.
- Compare different sampling methods (DDPM vs. DDIM).
- Visualize generated samples and sampling trajectories.
- Quantitatively evaluate sample quality with Wasserstein distances.

Datasets

We provide three toy datasets:

1. **1D 2-Gaussian mixture** (two modes).
2. **2D 8-Gaussian mixture** (points arranged in a ring).
3. **2D S-curve** (nonlinear manifold).

You can switch datasets via the provided data loader. All datasets are already normalized and scaled for you.

Model

The model is a simple multi-layer perceptron (MLP) with a small time embedding module. The code for MLP is fully provided. You do not need to modify the model or training code.

Training

Training is handled for you in the provided framework. You only need to implement the diffusion process correctly. We use default hyperparameters (number of steps $T = 500$, learning rate 2×10^{-3} , batch size 1024). With these defaults, you should already get good results without additional tuning.

Evaluation

We evaluate generated samples against the true data distribution:

- For **1D datasets**, we compute the **1-Wasserstein distance**.
- For **2D datasets**, we compute the **sliced Wasserstein-2 distance**.

The evaluation code is already provided and will run automatically after training and sampling.

Visualization

The plotting code is provided and will automatically generate:

- Distribution plots comparing real data and generated samples.
- Sample path plots (trajectories $x_T \rightarrow x_0$).

This will help you visualize and compare DDPM and DDIM sampling.

Your Implementation Task

You must complete the following methods inside the `GaussianDiffusion` class:

1. `__init__` Initialize diffusion hyperparameters:

- betas β_t , alphas $\alpha_t = 1 - \beta_t$.
- cumulative products $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.
- precomputed square roots needed for the forward process.

2. `q_sample(x0, t, noise)` Implement the forward (noising) process:

$$q(x_t | x_0) = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

3. `p_losses(x0, noise)` Sample random timesteps t , generate x_t using `q_sample`, predict ϵ with the model, and compute MSE loss against the true noise:

$$L = \mathbb{E}_{t, \epsilon} \left\| \epsilon - \epsilon_\theta(x_t, t) \right\|^2.$$

4. `sample_ddpm(n)` Implement sampling using the DDPM update rule:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I),$$

$$\text{with } \sigma_t^2 = \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t.$$

5. `sample_ddim(n, steps, eta)` Implement deterministic or stochastic DDIM sampling:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_\theta(x_t, t) + \sigma_t z,$$

$$\text{where } \hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \text{ and } \sigma_t \text{ depends on } \eta.$$

Deliverables

- Submit your completed `hw1.py` file to Gradescope.

Exploration

After finishing the required implementation, you are encouraged to:

- Experiment with different numbers of timesteps T .
- Try different values of `steps` and η for DDIM.
- Compare qualitative results of sampling paths.

Note

The assignment is designed so that with the given defaults you should get good results. However, feel free to **play with parameters** and observe the differences between DDPM and DDIM.